MISSION: MARSOPS_2036

WELCOME TO MARS.
PLEASE DON'T DIE!

MARSOPS: DISTRIBUTED AUTOMATION PLATFORM

INSTITUTION
Sapienza University of Rome

PROGRAM
MSc in Engineering in Computer Science & AI

GROUP
LoAP

PROJECT SCOPE

MarsOps is a distributed automation platform designed to guarantee the survival of occupants in a Martian habitat by integrating heterogeneous IoT devices into a unified monitoring system. The platform acts as a bridge between a fragile habitat's infrastructure and the human operators.

Operators can monitor critical environmental parameters in real-time through a centralized dashboard. They can, also, define rules to ensure that habitat actuators respond instantly to dangerous environmental shifts without manual intervention.

By evaluating sensor data against operator-defined criteria, it triggers autonomous responses, such as activating cooling systems or emergency ventilation, without requiring manual intervention, mitigating risks in high-latency Martian environments.

MISSION STATUS: OPERATIONAL

RELIABILITY

Distributed architecture ensuring habitat survival through redundancy.

AUTOMATION

Smart control loops mitigating risks in high-latency environments.

SCALABILITY

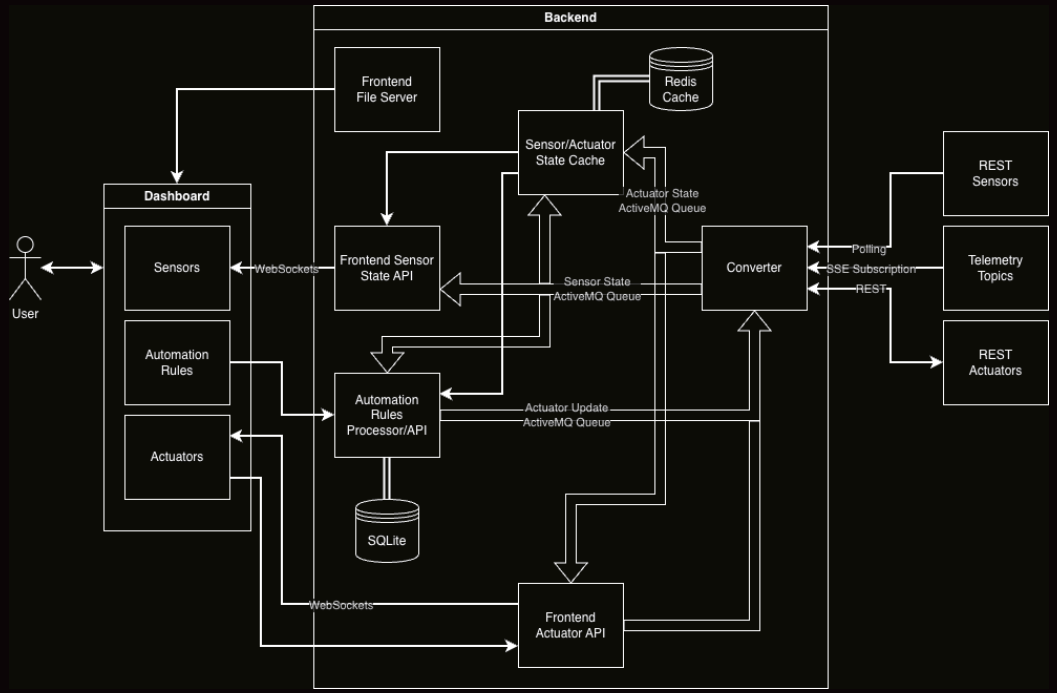
Microservices-ready design for future habitat expansions.

END OF TRANSMISSION

SYSTEM ARCHITECTURE

- MICROSERVICE ARCHITECTURE
- AMQ AS MESSAGE BROKER
- REDIS IN-MEMORY STATE CACHE
- REAL-TIME WEBSOCKET DASHBOARD
- DOCKER COMPOSE DEPLOYMENT
- REACTIVE RULE EVALUATION

SYSTEM ARCHITECTURE



INTERNAL ARCHITECTURE

REST POLLER

Polls 8 REST sensors every 5 seconds via httpx. Handles greenhouse_temperature, hydroponic_ph, co2_hall, air_quality, water_tank_level and more.

WEBSOCKET

Subscribes to real-time telemetry streams (e.g., power, environment, thermal, airlock) with auto-reconnect on disconnect.

NORMALIZER REGISTRY

Schema-driven registry: SENSOR_ID_NORMALIZERS for REST, SCHEMA_NORMALIZERS for SSE/actuators. Decouples source formats from unified schema.

COMMAND LISTENER

AMQP receiver on actuator.commands in a dedicated daemon thread. Bridges asyncio loop and AMQP via run_coroutine_threadsafe.

STATELESS DESIGN — NO DATABASE, NO PERSISTENT STORAGE. ALL STATE LIVES IN THE BROKER.

PURPOSE & CORE FUNCTIONALITY

AUTOMATED CONTROL LOOP

Implements the habitat's automated control loop.

RULE EVALUATION

Evaluates telemetry against operator-defined rules.

TELEMETRY PROCESSING

Continuously receives sensor telemetry from the message broker.

ACTUATOR COMMANDS

Emits commands for actuators when rule conditions are met.

OPERATIONAL EXAMPLE

If the temperature in a greenhouse group exceeds a configured threshold, a command is produced to switch the cooling fan ON. The process is automatic and requires no manual intervention.

CONVERTER SERVICE

The converter-service is the data ingestion and conversion layer of the MarsOps platform. It acts as a bridge between heterogeneous IoT sources, which have different event formats and communicate in different forms (such as SSE and polling), and the event queues that the other microservices attach to, ensuring that all other components operate on homogeneous and similarly structured data.

By normalizing the way to receive data from sensors, it deduplicates the other complexity while removing it from all the other microservices, ensuring that they can treat every source as basically identical as long as it provides the same “type” of metric (for example temperature, pressure or air particle concentration).

TARGET USER STORIES 1, 2, 3, 4, 5, 20

NORMALIZATION PIPELINE

REST SENSORS (8 SOURCES)

Poll every 5s via `httpx` --> `SENSOR_ID_NORMALIZERS` --> `UnifiedEvent` --> `sensor.events` (multicast)

TELEMETRY TOPICS (4 GROUPS)

SSE stream with auto-reconnect --> `SCHEMA_NORMALIZER` --> `UnifiedEvent` --> `sensor.events` (multicast)

ACTUATOR BOOT + COMMANDS

Fetch REST on startup + AMQP `actuator.commands` --> `ActuatorEvent` `actuator.states` (anycast)

UNIFIEDEVENT SCHEMA

`group_id` + `at` + `metrics[]{id, type, value[]}` + `status` + `converter_version`. Two event types, two queues.

Both REST and SSE sources converge into the same UnifiedEvent schema, eliminating source-specific complexity for all downstream consumers.

CONVERTER SERVICE - TECHNOLOGY STACK

PYTHON 3

Provides acceptable performance while being extremely fast to prototype in and having a rich ecosystem.

HTTPX

Enables HTTP request support in order to interface with the sensors' and actuators HTTP APIs.

AMQ (STOMP, PROTON)

Interacts with the message queues in order to exchange messages with every other part of the system.

CACHING SERVICE

IN-MEMORY STORAGE

The Caching Service uses Redis to store the most recent state of all sensors and actuators.

LOW LATENCY

Provides sub-millisecond access to habitat telemetry for critical decision-making.

STATE SYNCHRONIZATION

Ensures all microservices have a consistent view of the habitat's current status.

DASHBOARD SUPPORT

Powers the real-time dashboard by providing the initial state on connection.

TECHNICAL NOTE

By decoupling state from the primary database, the Caching Service reduces load on persistent storage and ensures high availability of real-time data across the entire MarsOps ecosystem.

CACHE SERVICE

The cache service ensures that upon outages of the converter service, or whenever other microservices restart, they can immediately query the system's current state without waiting for update messages to appear on the internal event queue. It does so by using Redis as an in-memory "ephemeral" cache, that however can still immediately provide the needed data.

This powers several parts of the system: for example, whenever the frontend connects for live updates, an initial batch of synthesized updates is immediately sent to notify it of the state that it should display, in order to never display an empty dashboard. Analogously, when the automation rules service starts, it can fetch the current state for all sensors and actuators so that it can know which rules to skip as they have already run.

CACHE SERVICE - TECHNOLOGY STACK

PYTHON 3

FASTAPI

Provides built-in support for OpenAPI and Swagger documentation, as well as enabling fast API prototyping.

REDIS

Provides the in-memory cache that powers the service.

METRICS SERVICE

The metrics service acts as middleware between the backend's internal event queues and cache and a user or frontend that might just want to receive live updates over more standard web browser protocols. For this reason, it exposes WebSockets for both general updates, spanning all metrics, and more granular updates, for specific metric "group", in order to avoid overloading a client with information that it doesn't care about.

As described in the previous slides, when a WebSocket is opened it first sends to it the currently cached state. This ensures that as soon as clients request data, they receive a valid copy of it to display, while still getting live updates later.

Additionally, it enables discovery of all available metrics and their names and info by the frontend, making sure that it doesn't have to hardcode almost anything.

ACTUATOR SERVICE

FUNCTION

Acts as the "bridge" between the Automation Engine (logic provider) and the physical actuators of the Martian habitat.

SUBSCRIPTION

Listens to the dedicated command topic (multicast/ actuator.commands) on AMQ/Kafka.

VALIDATION

Ensures the command matches the JSON schema defined in the contract.

EXECUTION

Provides a REST POST request to the specific actuator endpoint (e.g., `/api/actuators/cooling_fan`) to update its state. The new state is sent to the Queue `actuator.commands`, managed by the Converter Service who will then update the state of the actuator with a REST call to the Simulator.

REAL-TIME STATE BROADCASTING

Exposes dedicated WebSocket channels (e.g., `/api/actuators/ws`) to instantly push live state changes to the frontend. This ensures operators immediately see the physical response (ON/OFF) to their manual commands or automated rules without refreshing the dashboard.

ACTUATOR SERVICE

The actuator service also acts as middleware between the backend's internal event queues and cache and a user or frontend, however this time that is relative to the actuators' state and their updates. For this reason, in addition to the discovery features and WebSockets for live state updates, it also provides POST endpoints to manually toggle actuators from the dashboard.

Like the metrics service, it also caches actuators' state.

WEBSOCKETS

GLOBAL STATUS WEBSOCKET

`/api/sensors/ws & /api/actuators/ws`

Provides a broadcast stream of all sensor/actuator states, synchronizing the entire habitat dashboard upon connection by fetching the full dataset from the Redis cache.

TARGETED DEVICE WEBSOCKET

`/api/sensors/{id}/ws & /api/actuators/{id}/ws`

Dedicated low-latency channel for monitoring a specific hardware component, reducing overhead by filtering updates exclusively for the selected actuator ID.

METRICS/ACTUATOR SERVICES - TECHNOLOGY STACK

PYTHON 3

FASTAPI

AMQ (PROTON)

AUTOMATION RULES SERVICE

The automation rules service maintains a core part of the system, by managing and persisting the automation rules that make sure it correctly responds to specific situations detected by the sensors with the actuators it has available. For this reason, whenever it receives updates from the sensor state queue, it evaluates them against all the rules it has stored and decides if a rule needs to be fired. If so, it sends a message to the converter service to toggle the relevant actuator.

Internally, it uses SQLite to store automation rules, making sure that they don't go lost even if the system goes down. Additionally, it stores metadata about when the rules were fired, and why.

For a frontend, it provides a rich API to view, edit and delete all rules, as well as consulting the aforementioned logging data it has collected; like the other services, it also provides a description of all this in a standardized OpenAPI format.

AUTOMATION RULES - TECHNOLOGY STACK

JAVA 21 + LOMBOK + JACKSON

Industry-standard language, with libraries added specifically for boilerplate reduction.

SPRING BOOT 3.3.2

Spring Web, JMS, and JDBC modules.

SQLITE (JDBC)

Lightweight embedded persistent storage.

REST API & RULE DEFINITION

ENDPOINT: /API/RULES

Standard CRUD semantics for comprehensive rule management: list, get by ID, create, update, enable/disable, and delete rules.

GROUP_ID & METRIC_ID

Identifies metric within sensor group.

OPERATOR

Comparison operator (e.g., `lt`, `eq`, `gt`).

COMPARE_VALUE

Threshold for comparison (numeric or string).

UNIT

Optional unit filter for event value.

ACTUATOR_ID & ACTUATOR_STATE

Target actuator and boolean state.

ENABLED

Flag to activate/deactivate the rule.

VALIDATION & TRACEABILITY

Input validation is enforced via Jakarta Bean Validation (400 Bad Request for invalid inputs). All rule changes generate structured audit log entries prefixed with [AUTOMATION-AUDIT], ensuring critical configuration traceability.

RULE ENGINE & ANTI-SPAM LOGIC

EXECUTION PIPELINE

1. **EVENT PARSING**
   Extracting data from `MetricGroupStateEvent`.

2. **RULE MATCHING**
   Retrieving rules by `group_id` + `metric_id`.

3. **TYPED EVAL**
   Evaluation using `evaluateDouble()` or `evaluateString()`.

4. **COMMAND GENERATION**
   Creating structured `ActuatorCommand` object.

5. **PERSISTENCE**
   Recording firing in the `rule_firings` table.

SMART SUPPRESSION

If the actuator is already in the desired state, the command is suppressed to prevent broker flooding.

> `ActuatorStateListener maintains an in-memory ConcurrentHashMap of the current state of each actuator.`

MESSAGE SCHEMA

SENSOR EVENT

→ sensor.events

```
{
  "converter_version": "1.0.0",
  "group_id": "air_quality",
  "at": "2026-03...",
  "metrics": [{
    "id": "voc",
    "type": "...",
    "value": [{
      "value": 604.48,
      "unit": "ppb"
    }]
  }],
  "status": "ok|warning"
}
```

ACTUATOR STATE

→ actuator.states

```
{
  "actuator_id": "cooling_fan",
  "is_on": true,
  "updated_at": "2026-03-..."
}
```

ACTUATOR COMMAND

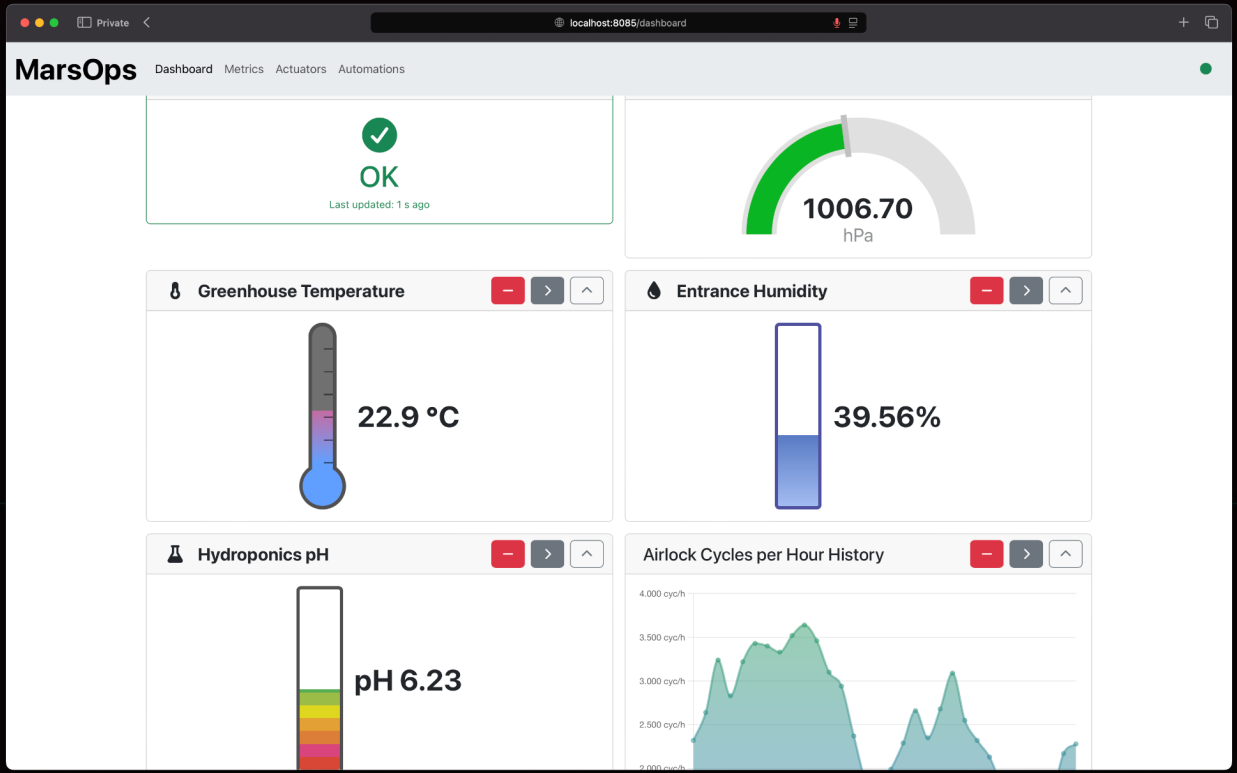
← actuator.commands

```
{
  "actuator_id": "cooling_fan",
  "is_on": true,
  "created_at": "2026-03-..."
}
```

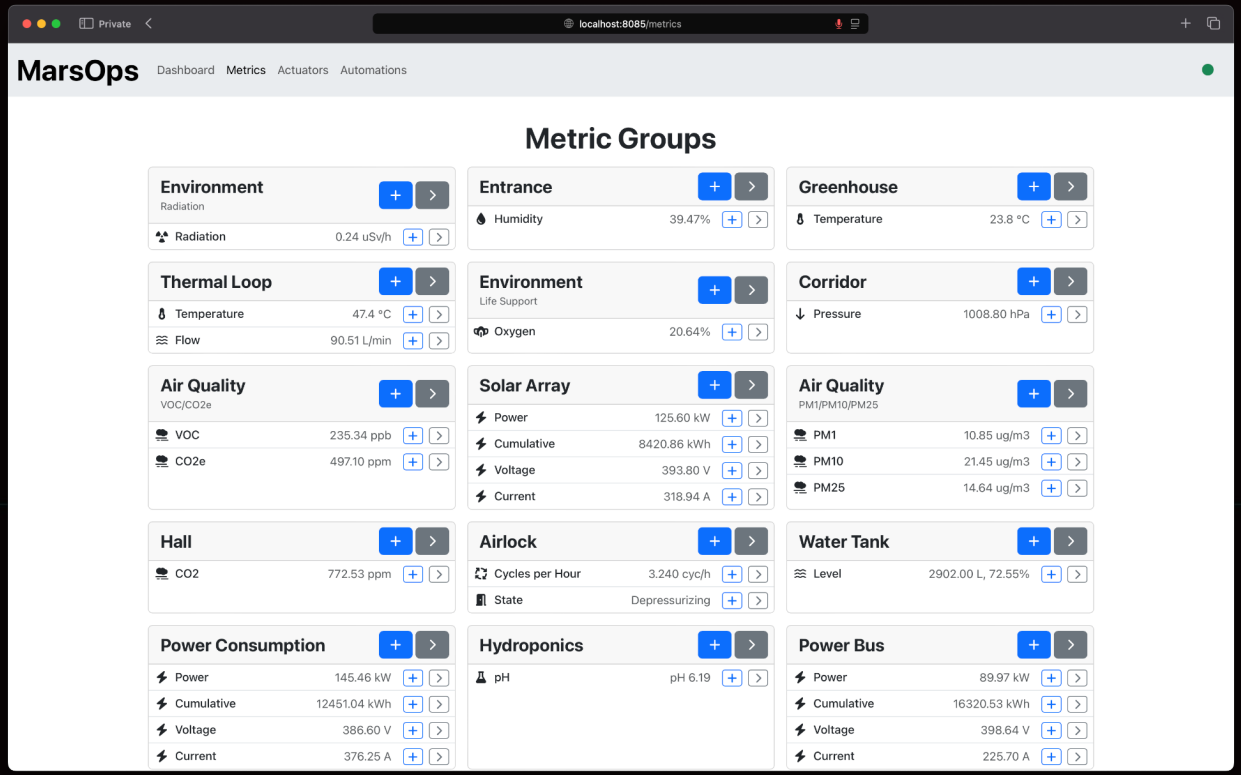
MESSAGING FLOW (END-TO-END)

Simulator REST API
↓ poll / fetch
Converter Service
↓ normalize
sensor.events (multicast)
↓ consume
automation-rules
↓ evaluate rules
actuator.commands (anycast)
↓ consume
Converter Service
↓ REST call
Simulator REST API
↓ publish new state
actuator.states (anycast)

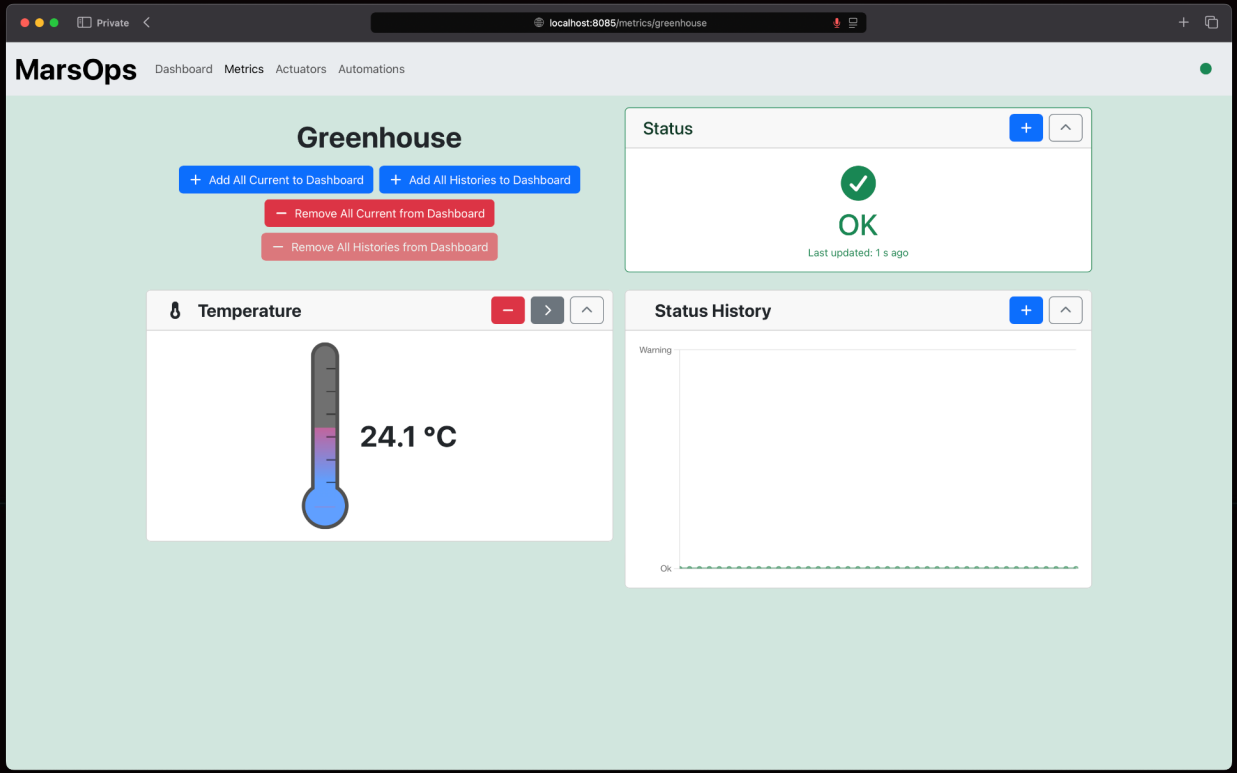
FRONTEND


Private
localhost:8085/dashboard
MarsOps
Dashboard
Metrics
Actuators
Automations
OK
Last updated: 1 s ago
1006.70
hPa
Greenhouse Temperature
22.9 °C
Entrance Humidity
39.56%
Hydroponics pH
pH 6.23
Airlock Cycles per Hour History
4.000 cyc/h
3.500 cyc/h
3.000 cyc/h
2.500 cyc/h

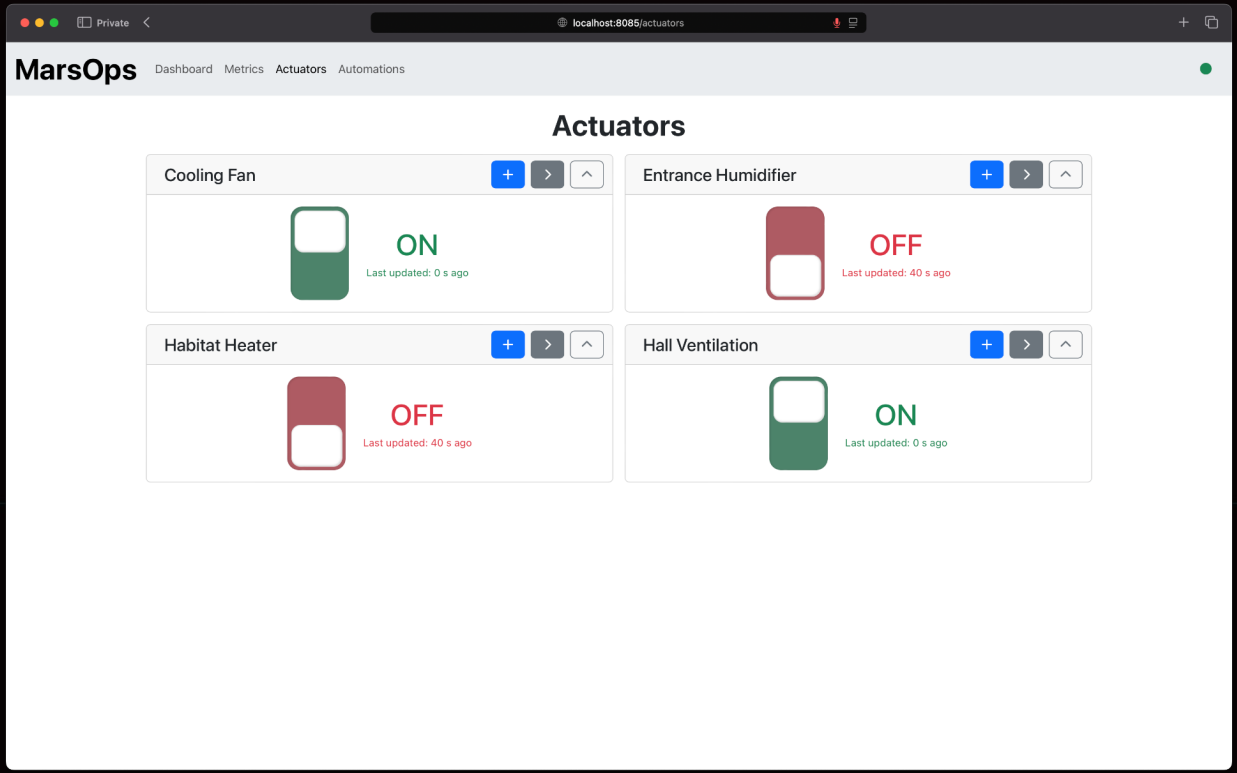
FRONTEND


Private
localhost:8085/metrics
MarsOps
Dashboard
Metrics
Actuators
Automations
Metric Groups
Environment
Radiation
Radiation 0.24 uSv/h
Entrance
Humidity 39.47%
Greenhouse
Temperature 23.8 °C
Thermal Loop
Temperature 47.4 °C
Flow 90.51 L/min
Environment
Life Support
Oxygen 20.64%
Corridor
Pressure 1008.80 hPa
Air Quality
VOC/CO2e
VOC 235.34 ppb
CO2e 497.10 ppm
Solar Array
Power 125.60 kW
Cumulative 8420.86 kWh
Voltage 393.80 V
Current 318.94 A
Air Quality
PM1/PM10/PM25
PM1 10.85 ug/m3
PM10 21.45 ug/m3
PM25 14.64 ug/m3
Hall
CO2 772.53 ppm
Airlock
Cycles per Hour 3.240 cyc/h
State Depressurizing
Water Tank
Level 2902.00 L, 72.55%
Power Consumption
Power 145.46 kW
Cumulative 12451.04 kWh
Voltage 386.60 V
Current 376.25 A
Hydroponics
pH pH 6.19
Power Bus
Power 89.97 kW
Cumulative 16320.53 kWh
Voltage 398.64 V
Current 225.70 A

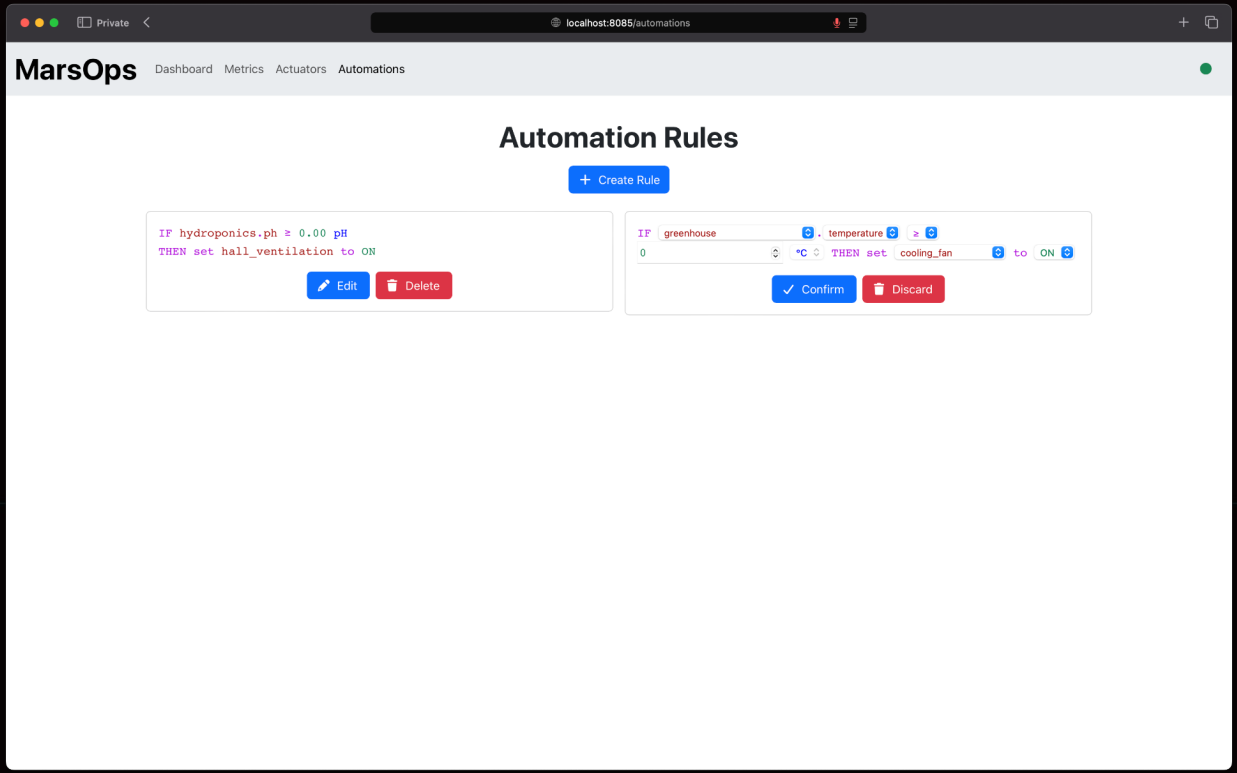
FRONTEND


Private
localhost:8085/metrics/greenhouse
MarsOps
Dashboard
Metrics
Actuators
Automations
Greenhouse
Add All Current to Dashboard
Add All Histories to Dashboard
Remove All Current from Dashboard
Remove All Histories from Dashboard
Status
OK
Last updated: 1 s ago
Temperature
24.1 °C
Status History
Warning
Ok

FRONTEND


Private
localhost:8085/actuators
MarsOps
Dashboard
Metrics
Actuators
Automations
Actuators
Cooling Fan
ON
Last updated: 0 s ago
Entrance Humidifier
OFF
Last updated: 40 s ago
Habitat Heater
OFF
Last updated: 40 s ago
Hall Ventilation
ON
Last updated: 0 s ago

FRONTEND


Private
localhost:8085/automations
MarsOps
Dashboard
Metrics
Actuators
Automations
Automation Rules
Create Rule
IF hydroponics.ph ≥ 0.00 pH
THEN set hall_ventilation to ON
Edit
Delete
IF greenhouse . temperature ≥
0 °C THEN set cooling_fan to ON
Confirm
Discard